GraphRAG 인덱싱 실습: 무엇을 만들고 어떻게 진행하나

RAG 실습의 청크 260개로 그래프(Neo4j)와 검색엔진 인덱스(OpenSearch)를 한 세대로 함께 만들고, 검증을 통과하면 게시함

만드는 것(교재 30쪽 워크플로우)

① 세대 ID 발급: gen-lab-{별명}-{해시 8자리}, 사전 검사

② 공통 단계: Chroma 청크에 chunk_id를 붙여 파일로 고정

③ KG 구축 K1 ~ K6, K8: 조항 나누기 → LLM 추출 → 적재 → 병합 → 설명 임베딩 → 커뮤니티

③ 검색엔진 S1 ~ S3: 임베딩 → 키워드 분석 → 문서 단위로 쓰기

④ 검증: 입력 = 검색엔진 = 그래프 청크 노드, 중복 없음

⑤ 게시: 활성 세대 파일 하나만 교체

수업 흐름(4H)

교시	내용	실습 함수
이론	교재 28 ~ 62쪽	-
A	세대, 청크, 추출 단위, 추출	5개
B	적재, 병합, 임베딩, 커뮤니티	6개
C	검색엔진, 검증, 게시, 확인	5개

실습 방식

- 표본 10청크(약관 2, 혜택 4, 상담 4)로 1분 안에 ① ~ ⑤ 완주
- 16개 함수(직접 6, 빈칸 10)를 my_graph.py에 채워 바꿔 끼움
- 전체 260청크는 강사 배포본을 재사용해 LLM 없이 만듦

검색엔진이 찾은 청크의 chunk_id와 같은 청크 노드가 그래프에 있어야 검색 결과에서 그래프로 건너감(교재 29쪽)

상세 보기 →

출처: 『신한카드 하이브리드AI_W05 ~ W07』 29 ~ 30쪽, teacher/plan/graphrag-indexing-개발계획서.md 4장

무단전재 및 배포 금지

사전 준비 5가지와 수업에서 쓰는 명령 2개

시작 검사가 키, Neo4j(APOC, GDS), OpenSearch, 벡터 인덱스 차원을 한 번에 확인하고 모자란 것을 알려 줌

사전 준비(수업 전주)

1. 저장소 pull, Python 3.13
2. RAG 실습 결과 rag/indexing/data/chroma 있음(없으면 강사 배포본)
3. Docker로 Neo4j와 OpenSearch 기동 (backing-services, healthy 확인)
4. graphrag 폴더에서 가상환경을 만들고 활성화, pip install -r requirements.txt
5. secrets/.env에 GROQ_API_KEY, COHERE_API_KEY

수업용 명령 2개(활성화 뒤 indexer 폴더에서)

python index_graph.py --run {별명}-s1 --sample

표본 10청크로 ① ~ ⑤ 완주. 약 1분, LLM 약 30번

python check_graph.py --qid Q09 --rerank

질문 하나로 검색엔진 → 청크 → 엔티티 → 커뮤니티 보고서 → 리랭크 → 적중 판정. 프롬프트에 (.venv)가 없으면 먼저 활성화

출력 줄	봐야 할 것	표본 실행 값(강사 PC)
[K1]	청크 → 단위 수	10 → 18
[K5]	합친 묶음 수	4(상담 3, 상품 안내 1)
④ 검증	검색엔진 n/n, 청크 노드, 커뮤니티	10/10, 10 = 10/10, 13개

> ***같은 명령을 다시 돌리면 ① 사전 검사가 「할 일 없음」으로 끝남. 다시 만들려면 --full-reindex와 새 이름***

상세 보기 →

출처: hands-on/solutions/graphrag/indexer/README.md 3 ~ 5장, 2026-10-24 강사 PC 실측

무단전재 및 배포 금지

교시 A: 세대, 청크, 추출 단위, 추출(실습 함수 5개)

개념이 순수 파이썬 논리에 있으면 직접 작성, 라이브러리나 DB 호출 순서에 있으면 빈칸 채우기로 둠

단계	함수	방식	채울 것(실습 개념)	교재
①	issue_generation	직접	세대 ID gen-lab-{이름}-{해시 8자리}, 사전 검사 조건 3개를 and로	31
②	prepare_chunks	빈칸 2	doc_id 규칙(상담은 record_id, 나머지는 파일 이름 해시), chunk_id 규칙	31
K1	split_by_heading	직접	제목 줄 + 다음 줄 ① 조건으로 시작점, 다음 시작점 전까지 한 조각	34
K1	k1_split_units	빈칸 2	단위 ID = 문서 ID + # + 조항 번호, 겹친 조항 합치기	34
K2 K3	extract_one	빈칸 2	TextChunk(uid=단위 ID)를 추출기에, 결과를 바로 가지치기	36

교시 끝에 확인할 출력

- [K1] 청크 10 → 단위 18(약관 10, 혜택 4, 상담 4), 겹침 0
- [K2] LLM 18번(두 번째부터 0번, 재사용 18)
- [K3] 실패 단위 0, 단위 하나의 노드와 관계 이름

방식을 고른 기준

- 직접 작성: 순수 파이썬 논리(규칙, 비교, 조립)가 개념인 단계
- 빈칸 채우기: 호출 순서가 개념인 단계
- 교재에 코드가 있는 자리만 빈칸

> ***이 실습 데이터는 쪽 단위 청킹이라 두 청크에 걸친 조항이 없음(겹침 0). 합치기 코드는 그대로 두고 README에 이유를 적음***

상세 보기 →

출처: hands-on/solutions/graphrag/indexer/core_graph.py, teacher/plan/graphrag-indexing-개발계획서.md 4장

무단전재 및 배포 금지

교시 B: 적재, 병합, 설명 임베딩, 커뮤니티(실습 함수 6개)

그래프를 쓰는 단계는 모두 Cypher나 라이브러리 호출이 개념이라 빈칸 채우기, 설명 글 만들기만 직접 작성임

단계	함수	방식	채울 것(실습 개념)	교재
K4	build_graph_rows	빈칸 2	청크마다 Chunk 노드, 엔티티마다 나온 청크로 FROM_CHUNK	38
K5	k5_merge	빈칸 2	같은 세대에서 값이 같은 노드 모으기, apoc.refactor.mergeNodes	41
K6	entity_description	직접	관리용 속성 빼고 이름순 「라벨: k=v, …」	43
K6	k6_embed_descriptions	빈칸 2	setNodeVectorProperty로 저장, 벡터 인덱스 생성	44
K8	detect_communities	빈칸 2	gds.graph.project(무방향), gds.leiden.stream(씨앗 42, 병렬 1)	48
K8	build_community_rows	빈칸 2	3개 이상 묶음만, 구성원 번호 → 엔티티 ID	49

교시 끝에 확인할 출력

- [K4] 노드 약 100(청크 노드 10), 관계 약 150
- [K5] 4묶음(상담 3, 상품 안내 1)
- [K6] 임베딩 약 80개, 2048차원
- [K8] 묶음 16개 중 13개 요약, ④ 검증(K8) 통과

보는 포인트

- FROM_CHUNK 수 = 엔티티 수
- 병합 뒤 상품 안내 노드 1개(청크 4개분 합침)
- 설명 글 「Proposal: content=…, decision=수용」

K5 병합은 스키마의 고유 속성(ProductGuide.product_id, Consultation.record_id)만 기준으로 하고 다른 라벨은 합치지 않음

상세 보기 →

출처: hands-on/solutions/graphrag/indexer/core_graph.py, 교재 37 ~ 49쪽

무단전재 및 배포 금지

교시 C: 검색엔진 인덱싱, 검증, 게시(실습 함수 5개)

벡터 재사용, 검증, 게시는 순수 파이썬 논리라 직접 작성이고, 검색엔진 쓰기만 빈칸 채우기임

단계	함수	방식	채울 것(실습 개념)	교재
S1	embed_with_reuse	직접	해시가 옛 벡터에 없는 글만 임베딩, 원래 순서로 합침	55
S2	build_search_docs	빈칸 2	lexicon.tokenize, 엔진 문서 한 건(text_tokens는 공백으로 이음)	53, 58
S3	apply_search	빈칸 3	delete_by_query(doc_id), bulk(_id = chunk_id), refresh	59
④	verify_chunk_ids	직접	집합 비교 2개 + 목록 그대로 세어 중복 노드 찾기	61
⑤	publish_active	직접	임시 파일에 다 쓴 뒤 os.replace로 바꿔 끼움	30, 62

교시 끝에 확인할 출력

- [S1] 두 번째 실행부터 재사용 10, 새로 0
- [S2] 카드명 1개, 별칭 18개
- [S3] 문서 6개, 10건 넣음
- ④ 검증 통과 10/10, ⑤ 게시 활성 세대

완주 뒤

- USE_TEACHER_CODE가 비면 본인 코드만으로 돎
- check_graph.py --qid Q09 로 본인 세대 확인
- 조 간 코드 리뷰: 직접 작성 6개를 서로 읽음

문서 단위로 지우고 넣어야 고친 문서의 옛 청크가 남지 않음(교재 58쪽). 청크 단위로 덮어쓰면 고아 청크가 검색에 섞임

상세 보기 →

출처: hands-on/solutions/graphrag/indexer/core_graph.py, 교재 53 ~ 62쪽

무단전재 및 배포 금지

본인 코드로 바꿔 끼우기: import 한 줄과 USE_TEACHER_CODE 목록

안 채운 함수는 강사 코드가 대신 돌므로 한 함수씩 채우며 전체를 끝까지 돌려 볼 수 있음

교체 절차

1. index_graph.py의 import 한 줄을 import my_graph as core 로 바꿈
2. my_graph.py 맨 아래 USE_TEACHER_CODE 목록에서 지금 채울 함수 이름을 지움
3. 그 함수를 채우고 새 이름으로 실행: --run {별명}-s2 --sample --full-reindex
4. 출력의 수를 강사 코드 실행과 비교함(단위 18, 병합 4, 검증 10/10)
5. 막히면 이름을 목록에 다시 넣고 다음 함수로 넘어감

완료 기준

- USE_TEACHER_CODE가 빈 my_graph.py로 표본 실행이 ④ 통과, ⑤ 게시
- [K1] 단위 18개, [K5] 4묶음, ④ 10/10과 커뮤니티 13개가 강사 결과와 같음(노드 수는 ±10 허용)
- check_graph.py --qid Q09가 청크와 엔티티 둘 다 적중
- 제출: my_graph.py와 마지막 실행 화면 캡처

막혔을 때

- NotImplementedError 빈칸 N: 아직 안 채움 또는 raise 줄을 안 지움
- 세대 …는 지금 활성 세대임: --run 이름을 바꿈
- 429로 K2가 느림: 기다리거나 --reuse data/teacher
- 중단 뒤 재실행: 같은 --run이면 ①이 흔적을 지우고 다시 만듦

교시 A가 끝나면 뒤 단계는 강사 코드로 돌아 전체가 한 번 완주됨. 앞 교시에서 막혀도 다음 교시 실습이 가능함

상세 보기 →

출처: hands-on/solutions/graphrag/indexer/my_graph.py, README.md 6장과 9장

무단전재 및 배포 금지